Sugar Desk & Sugar Module — Developer Architecture & Technical Integration Guide

Document Version: 2.0
Target Audience: Full-Stack Developers, Frappe/ERPNext Engineers, Frontend Developers
Author: CBD IT Solutions Pvt. Ltd.
System Stack: Frappe Framework v16 (Python / MariaDB) + Vue 3 (Vite / Single Page Application)

---

1. Executive System Overview

The **Sugar Module** is an enterprise-grade ERP solution tailored specifically for the Indian Sugar Trading ecosystem. It merges the **backend data integrity and accounting rigor of Frappe Framework v16** with the **high-speed, keyboard-first UX of Tally ERP**.

Key Architectural Pillars:

1. **Headless SPA Frontend**: Built with Vue 3 and Vite, compiled into static bundles served seamlessly via Frappe's web router (`/sugar-desk`).
2. **RESTful RPC Backend**: A clean Python API layer (`sugar_module.sugar_module.api`) that wraps MariaDB transactional DocTypes into high-speed, validated endpoints.
3. **Reactive Keyboard Engine**: Singleton keystroke interceptor mapping standard Tally shortcuts (`P`, `D`, `Y`, `R`, `T`, `M`, `V`, `Ctrl+G`, `F1`–`F12`) while isolating input elements to prevent keystroke bleeding.
4. **Universal Database Search**: Cross-DocType spotlight search engine indexing purchase lots, dispatch deliveries, payments, receipts, UTRs, vehicle numbers, mill suppliers, brokers, and customer accounts.

---

2. High-Level Architecture & Communication Flow

```
|      +---> save_voucher()              --> Dynamic Doc Creation & Validation      |
|      +---> get_register_data()         --> Paginated Multi-Criteria Query Builder |
|      +---> get_day_book()              --> Unified Chronological Ledger           |
|      +---> search_ledgers_and_parties()--> Typeahead Ledger Autocomplete         |
|                                        |                                          |
+----------------------------------------|------------------------------------------+
                                         |
                                    SQL / ORM
                                         |
+----------------------------------------v------------------------------------------+
|                          MARIADB DATABASE (sugar.local)                           |
|                                                                                   |
|  tabSugar Purchase | tabDispatch Entry | tabPurchase Payment | tabBroker Party Payment  |
|  tabSupplier       | tabBroker         | tabCustomer         | tabItem | tabAccount  |
+-----------------------------------------------------------------------------------+
```

---

3. Detailed Frontend-to-Backend Function Calling Mechanism

3.1 The Communication Pipeline

All frontend communications are centralized in `/src/composables/useFrappeApi.js`. This composable encapsulates: 1. **CSRF Token Handling**: Extracts `window.csrf_token` or reads cookies provided by Frappe sessions. 2. **Method Routing**: Automatically constructs the Frappe RPC URL `/api/method/<python_module_path>.<function_name>`. 3. **Payload Serialization**: Converts Javascript objects into JSON or query parameters. 4. **Response Unwrapping**: Standard Frappe responses are structured as `{"message": <payload>}`. The composable unwraps this layer cleanly.

3.2 Code Example: Invoking Backend from Vue 3

```
// frontend/src/composables/useFrappeApi.js
export function useFrappeApi() {

  // Generic RPC Caller
  const call = async (method, args = {}) => {
    try {
      const res = await fetch(`/api/method/${method}`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'X-Frappe-CSRF-Token': window.csrf_token || ''
        },
        body: JSON.stringify(args)
      })
      if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`)
      const data = await res.json()
      return data.message !== undefined ? data.message : data
    } catch (err) {
      console.error(`[FrappeApi] Call to ${method} failed:`, err)
      throw err
    }
  }

  // Example: Saving a Voucher
  const saveVoucher = async (doctype, doc) => {
    return await call('sugar_module.sugar_module.api.save_voucher', {
      doctype: doctype,
```

```
      doc: JSON.stringify(doc)
    })
  }

  // Example: Universal Global Search
  const universalGlobalSearch = async (query = '') => {
    const res = await fetch(`/api/method/
sugar_module.sugar_module.api.universal_global_search?query=${encodeURIComponent(query)}
&limit=30`)
    if (res.ok) {
      const data = await res.json()
      return data.message || []
    }
    return []
  }

  return { call, saveVoucher, universalGlobalSearch, /* ... */ }
}
```

---

4. Comprehensive Python Backend API Specification (`api.py`)

All backend APIs are defined in `/home/frappe/frappe-bench-v16/apps/sugar_module/sugar_module/sugar_module/api.py`.

4.1 `get_gateway_metrics(period)`

- **Purpose**: Computes 100% real-time KPI metrics for the Gateway dashboard.
- **Parameters**: `period` (str: `"Today"` | `"MTD"`).
- **Backend Logic**:
  - Calculates date bounds (`today` vs start of month `YYYY-MM-01`).
  - Queries MariaDB tables:
    - `Sugar Purchase`: opening stock, today's purchase qty/val, current available stock (`available_qty_quintal`).
    - `Dispatch Entry`: total sales qty, sales value, balance amount (Receivable).
    - `Purchase Payment`: total disbursements made to sugar mills.
    - `Broker Party Payment`: total receipts received from customer buyers.
- **Returns**:

```
{
  "opening_stock_qty": 6290.0,
  "today_purchases_qty": 0.0,
  "total_sales_qty": 3440.0,
  "closing_stock_qty": 6290.0,
  "purchases_val": 35272000.0,
  "sales_val": 14816000.0,
  "payments_received": 3680000.0,
  "payments_made": 27600000.0,
  "total_receivable": 1852000.0,
  "total_payable": 7672000.0
}
```

4.2 `universal_global_search(query, limit)`

- **Purpose**: High-speed, multi-table spotlight search engine.

- **Parameters**: `query` (str), `limit` (int, default: 25).
- **Entities Searched**:
    1. **System Views & Reports**: Gateway, Vouchers (Purchase, Dispatch, Payment, Receipt, Contra), Registers, Day Book, Outstandings.
    2. **Sugar Purchase Vouchers**: Searched by ID (`PUR-2026-00001`), Supplier Mill, Item Grade.
    3. **Dispatch Entry Vouchers**: Searched by ID (`DIS-ENT-2026-00001`), Customer Party, Broker, Vehicle Number (`MH19CZ1234`), Sugar Purchase link.
    4. **Purchase Payments**: Searched by ID, Supplier, UTR Number, Reference No.
    5. **Broker Party Payments**: Searched by ID, Customer, Broker, UTR No.
    6. **Masters**: Suppliers, Brokers (with city & mobile), Customers, Items, Accounts.
- **Returns**: An array of structured result objects containing `title` , `subtitle` , `category` , `icon` , `route` , `doctype` , and `voucherId` .

4.3 `save_voucher(doctype, doc)`

- **Purpose**: Creates, validates, and submits vouchers to MariaDB.
- **Parameters**:
    - `doctype` (str): `"Sugar Purchase"` , `"Dispatch Entry"` , `"Purchase Payment"` , `"Broker Party Payment"` .
    - `doc` (str | dict): Complete form payload.
- **Validation Engine**:
    - **Stock Check**: On `Dispatch Entry` , verifies that `dispatch_qty_quintal` does not exceed the remaining `available_qty_quintal` of the linked `Sugar Purchase` .
    - **Party Resolution**: Resolves textual mill, broker, and customer names to valid database IDs using `resolve_link()` .
    - **Auto-Calculations**: Computes totals, taxes, paid amount, and remaining balances.
    - **Status & Workflow**: Sets status to `Submitted` (`docstatus = 1`).

4.4 `get_day_book(date, period)`

- **Purpose**: Merges all financial and stock transactions into a single unified audit ledger.
- **Parameters**: `date` (str: `YYYY-MM-DD`), `period` (str: `"Day"` | `"Week"` | `"Month"` | `"Year"`).
- **Entities Combined**:
    - `Sugar Purchase` (Type: Purchase, Voucher: `P`)
    - `Dispatch Entry` (Type: Dispatch/Sales, Voucher: `D`)
    - `Purchase Payment` (Type: Payment, Voucher: `Y`)
    - `Broker Party Payment` (Type: Receipt, Voucher: `R`)
- **Returns**: Chronologically sorted transaction list with running balances, counter-parties, debit/credit values, and status tags.

---

5. Keyboard Engine & Tally Navigation Architecture

5.1 Singleton Keydown Listener (`useKeyboardEngine.js`)

To achieve authentic Tally keyboard operation without double-firing, a **Singleton Global Listener** is initialized on the `window` object.

```
 // frontend/src/composables/useKeyboardEngine.js
window.addEventListener('keydown', (e) => {
  const targetTag = (e.target?.tagName || '').toUpperCase()
  const activeTag = (document.activeElement?.tagName || '').toUpperCase()
  const inInput = ['INPUT', 'TEXTAREA', 'SELECT'].includes(targetTag) ||
```

```
                ['INPUT', 'TEXTAREA', 'SELECT'].includes(activeTag)

  // 1. Overlay triggers: Alt+G (Go To) or Ctrl+G (Global Search)
  if (e.altKey && (e.key === 'g' || e.key === 'G')) {
    e.preventDefault()
    globalUiState.isGoToOpen.value = !globalUiState.isGoToOpen.value
    return
  }

  // 2. CRITICAL ISOLATION: When user is typing inside an input field,
  // DO NOT intercept single character hotkeys (P, D, Y, R, T, M, V)
  if (inInput) {
    return
  }

  // 3. Navigation Hotkeys outside inputs
  const key = e.key.toUpperCase()
  if (key === 'P') { router.push('/voucher/purchase'); return }
  if (key === 'D') { router.push('/voucher/dispatch'); return }
  if (key === 'Y') { router.push('/voucher/payment'); return }
  if (key === 'R') { router.push('/voucher/receipt'); return }
  if (key === 'T') { router.push('/voucher/contra'); return }
  if (key === 'M') { globalUiState.activeSidebarCategory.value = 'MASTERS'; return }
  if (key === 'V') { globalUiState.activeSidebarCategory.value = 'VOUCHERS'; return }
})
```

5.2 Hotkey Mapping Table

Shortcut Key	Function / Destination	Target Route
`P`	Sugar Purchase Voucher	`/voucher/purchase`
`D`	Dispatch Entry Voucher	`/voucher/dispatch`
`Y`	Purchase Payment Entry	`/voucher/payment`
`R`	Broker Party Receipt Entry	`/voucher/receipt`
`T`	Contra / Bank Transfer	`/voucher/contra`
`M`	Filter Sidebar to Masters	Activates Masters Tab
`V`	Filter Sidebar to Vouchers	Activates Vouchers Tab
`Ctrl+G`	Focus Universal Global Search	Focuses Search Input
`Alt+G`	Open "Go To" Command Palette	Opens Modal
`F10` or `B`	Open Day Book Ledger	`/daybook`
`Esc`	Return to Gateway Dashboard	`/`

---

6. End-to-End Execution Traces

Trace 1: Saving a Dispatch Entry Voucher

```
 User presses 'D'
    |
    v
Router navigates to VoucherEntryView.vue (type='dispatch')
    |
    v
User fills Form (Party: Shree Traders, Lot: PUR-2026-0001, Qty: 100 Qtl, Rate: 3950)
    |
    v
User presses Enter on last field / clicks 'Save Voucher'
    |
    v
Frontend calls: saveVoucher('Dispatch Entry', form)
    |
    v
HTTP POST /api/method/sugar_module.sugar_module.api.save_voucher
    |
    v
Backend api.py:
    1. Validates available qty on PUR-2026-0001 (e.g. 500 Qtl >= 100 Qtl -> OK)
    2. Resolves 'Shree Traders' -> Customer ID 'CUST-00012'
    3. Computes Total Amount: 100 * 3950 = Rs. 3,95,000
    4. Creates new Dispatch Entry doc and sets docstatus=1 (Submit)
    5. Deducts 100 Qtl from Sugar Purchase available_qty_quintal (now 400 Qtl)
    |
    v
MariaDB Transaction Committed
    |
    v
Response 200 OK: {"message": {"name": "DIS-ENT-2026-00042", "status": "Submitted"}}
    |
    v
Frontend displays Success Toast + Switches to Read-Only Mode + Offers Print Voucher
```

---

7. Directory Structure & File Mapping

```
 frappe-bench-v16/apps/sugar_module/
├── frontend/                              # Vue 3 SPA Codebase
│   ├── index.html                         # HTML Entry with CBD Logo Favicon
│   ├── vite.config.js                     # Vite Bundler Configuration
│   ├── src/
│   │   ├── main.js                        # Vue App Mount & Global Plugins
│   │   ├── App.vue                        # Root Layout (TopBar + RouterView)
│   │   ├── style.css                      # Tally Navy Theme & Design Tokens
│   │   ├── assets/
│   │   │   └── logo.png                   # Official CBD IT Solutions Brand Logo
│   │   ├── router/
│   │   │   └── index.js                   # Vue Router Definitions
│   │   ├── composables/
│   │   │   ├── useFrappeApi.js            # Centralized Backend RPC Client
│   │   │   ├── useKeyboardEngine.js       # Global Keyboard & Hotkey Engine
│   │   │   ├── useExport.js               # Excel, CSV, PDF & Print Services
```

```
│   │   │   └── useTheme.js                 # Dark / Light Theme Manager
│   │   ├── components/
│   │   │   └── common/
│   │   │       ├── TopBar.vue              # Top Navigation Ribbon with CBD Logo
│   │   │       ├── MenuPanel.vue           # Tally Right Sidebar (Underlined Hotkeys)
│   │   │       ├── LedgerDropdown.vue      # High-Speed Autocomplete Dropdown
│   │   │       └── GoToPalette.vue         # Command Palette (Alt+G)
│   │   └── views/
│   │       ├── GatewayView.vue             # Dashboard with Top Search & 10 Metric Cards
│   │       ├── VoucherEntryView.vue        # Fast Keyboard Voucher Entry Form
│   │       ├── RegisterView.vue            # Data Registers (Purchase, Dispatch,
Outstandings)
│   │       ├── DayBookView.vue             # Chronological Day Book Audit Ledger
│   │       └── MastersView.vue             # Directory of Mills, Brokers, Buyers, Items
│
└── sugar_module/                           # Python Backend App
    ├── hooks.py                            # Frappe App Registration & Routes
    ├── www/
    │   ├── sugar_desk.py                   # Context Provider for /sugar-desk
    │   └── sugar_desk.html                 # Jinja Wrapper for compiled Vue Bundle
    ├── public/
    │   ├── logo.png                        # Public Logo Asset
    │   └── frontend/                       # Target Production Build Directory
    │       ├── index.html                  # Compiled Entry
    │       └── assets/                     # Bundled JS, CSS, and Images
    └── sugar_module/
        ├── api.py                          # All Whitelisted RPC Methods
        └── doctype/                        # MariaDB Schema Definitions
            ├── sugar_purchase/             # Sugar Purchase Lot Schema
            ├── dispatch_entry/             # Dispatch & Delivery Schema
            ├── purchase_payment/           # Mill Payment Schema
            └── broker_party_payment/       # Customer Receipt Schema
```

---

8. Build, Deployment & Developer Commands

Compile Frontend for Production:

```
cd /home/frappe/frappe-bench-v16/apps/sugar_module/frontend
npm run build
```

Ensure Ownership & Clear Frappe Cache:

```
chown -R frappe:frappe /home/frappe/frappe-bench-v16/apps/sugar_module
bench --site sugar.local clear-cache
```

Git Version Control:

```
cd /home/frappe/frappe-bench-v16/apps/sugar_module
git add -A
git commit -m "feat: <description>"
git push upstream sugar-module
```

End of Developer Architecture & Technical Integration Guide.